Gaming Agents Paper Presentation

Chuxuan Hu, Dwip Dalal, Xiaona Zhou
Group 5

CS598: Topics in LLM Agents, 2025 Spring

Voyager: An Open-Ended Embodied Agent with Large Language Models

Guanzhi Wang[1 2]✉, **Yuqi Xie**[3], **Yunfan Jiang**[4*], **Ajay Mandlekar**[1*],
Chaowei Xiao[1 5], **Yuke Zhu**[1 3], **Linxi "Jim" Fan**[1†]✉, **Anima Anandkumar**[1 2†]
[1]NVIDIA, [2]Caltech, [3]UT Austin, [4]Stanford, [5]UW Madison
*Equal contribution †Equal advising ✉ Corresponding authors
https://voyager.minedojo.org

Voyager

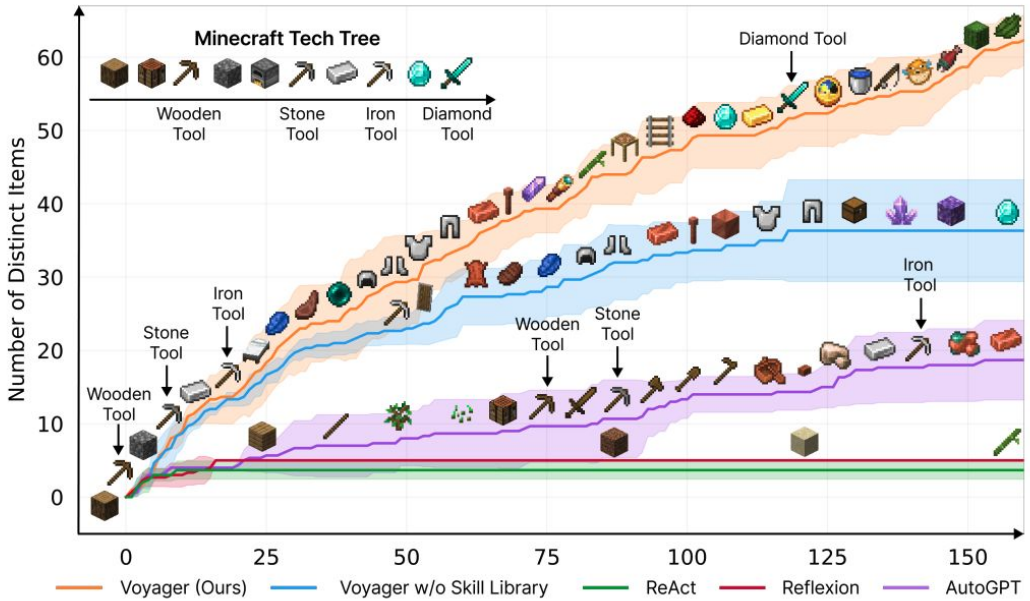


Figure 1: VOYAGER discovers new Minecraft items and skills continually by self-driven exploration, significantly outperforming the baselines. X-axis denotes the number of prompting iterations.

- LLM-powered lifelong learning agent in Minecraft

Background: Generalist Agents

- Challenges in Building Generalist Agents
  - Need to generalize across tasks in open-ended, long-horizon environments.
  - Traditional RL agents often:
    - Rely on sparse rewards,
    - Reset after each episode or task,
    - Lack mechanisms for lifelong learning or skill reuse.

Background: Baselines

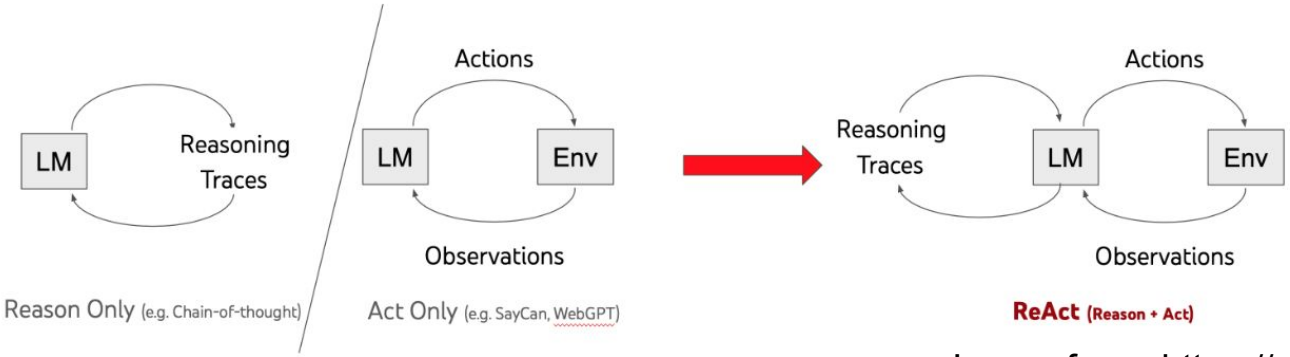
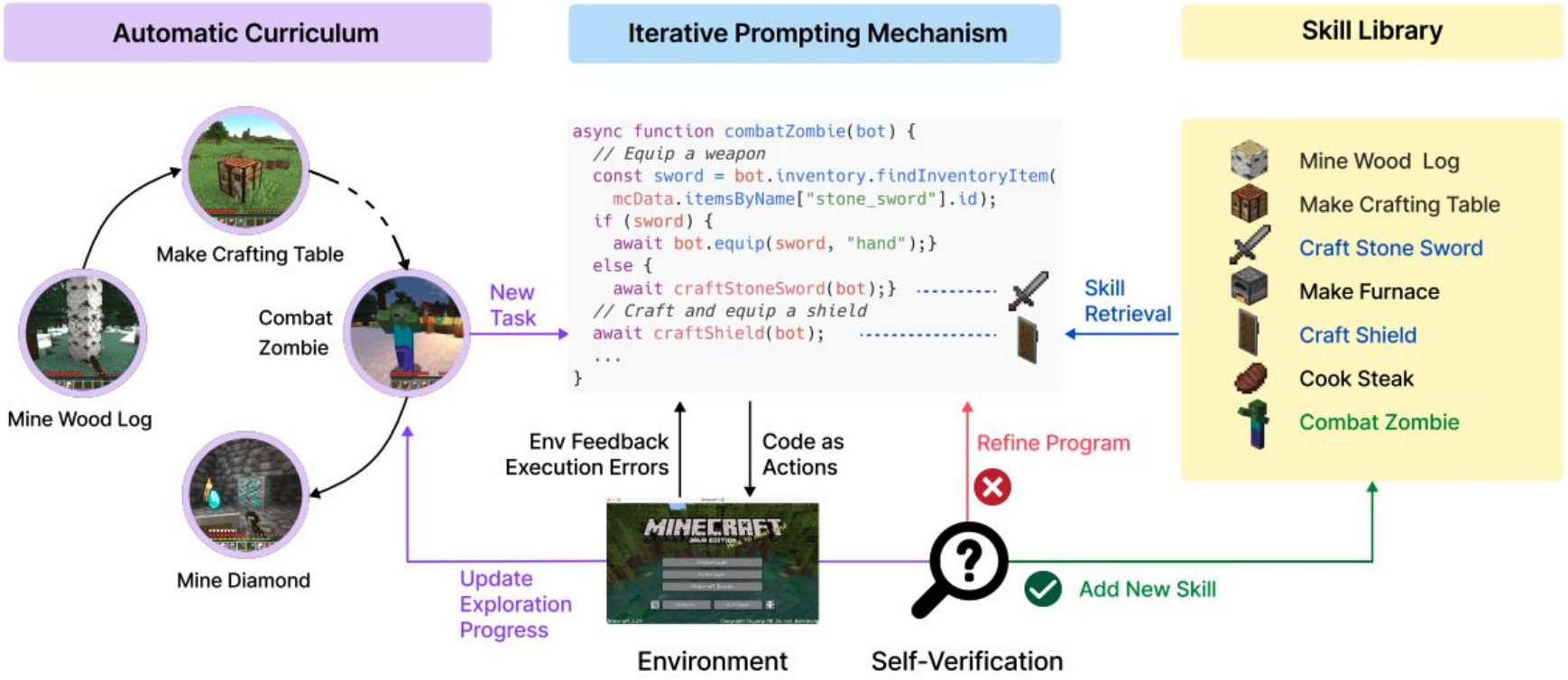


Image from: https://react-lm.github.io/

- ReAct: Combines **chain-of-thought** reasoning with **action execution** in an environment
- Reflexion: Builds on ReAct by adding **self-reflection**:
- AutoGPT: Decomposes a high-level goal into **subtasks**, uses a looped ReAct-like structure to manage planning, acting, and updating the plan.

Voyager: Components



Voyager: Automatic Curriculum Generator

- Generated by GPT-4 based on the overarching goal of “discovering as many diverse things as possible”

Figure 3: Tasks proposed by the automatic curriculum. We only display the partial prompt for brevity. See Appendix, Sec. A.3 for the full prompt structure.

Voyager: Automatic Curriculum Generator

- Generated by GPT-4 based on the overarching goal of "discovering as many diverse things as possible"

(1) **Directives encouraging diverse behaviors and imposing constraints**, such as "`My ultimate goal is to discover as many diverse things as possible ... The next task should not be too hard since I may not have the necessary resources or have learned enough skills to complete it yet.`";

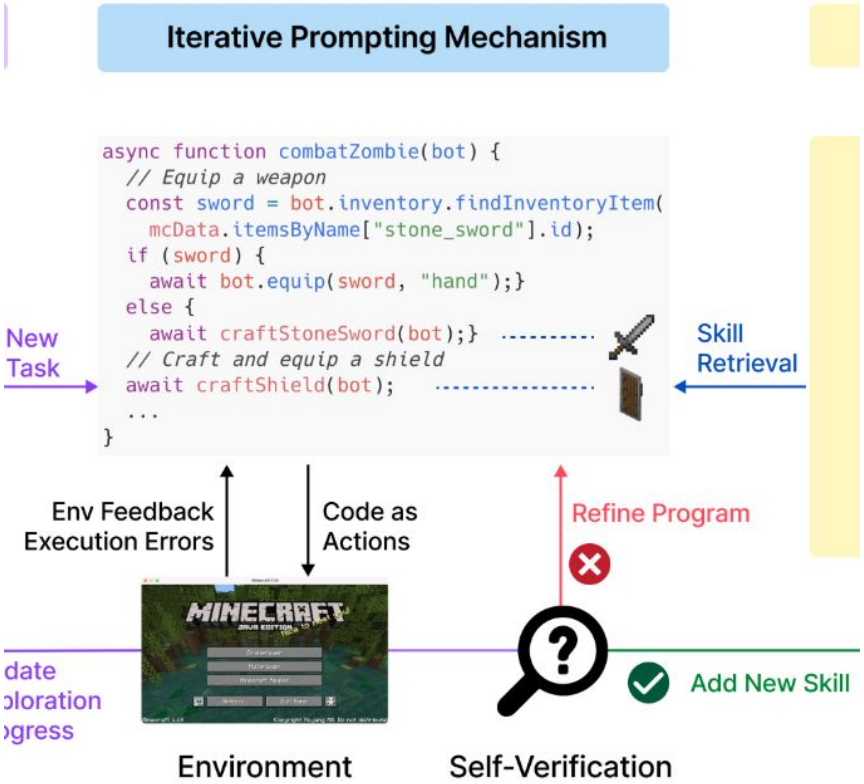
(2) **The agent's current state**, including inventory, equipment, nearby blocks and entities, biome, time, health and hunger bars, and position;

(3) **Previously completed and failed tasks**, reflecting the agent's current exploration progress and capabilities frontier;

(4) **Additional context**: We also leverage GPT-3.5 to self-ask questions based on the agent's current state and exploration progress and self-answer questions. We opt to use GPT-3.5 instead of GPT-4 for standard NLP tasks due to budgetary considerations.

With additional context from wiki knowledge base

Voyager: Iterative Prompting



Voyager: Iterative Prompting

- **Environment Feedback:** Mitigate Lack of Environmental Grounding
- **Execution Error:** inform GPT about which functions are available or missing

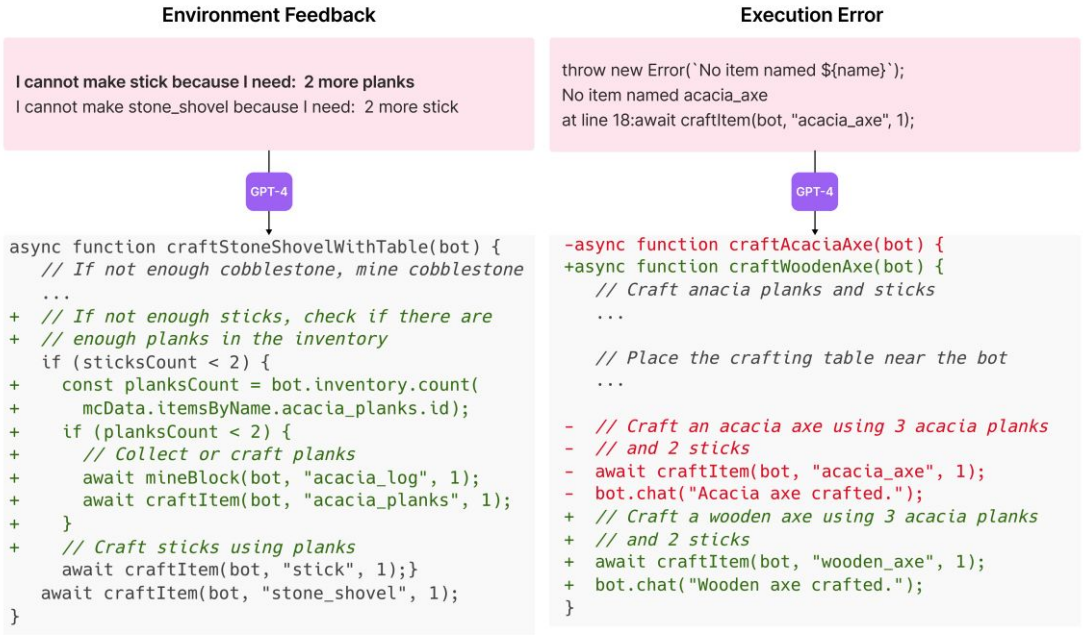
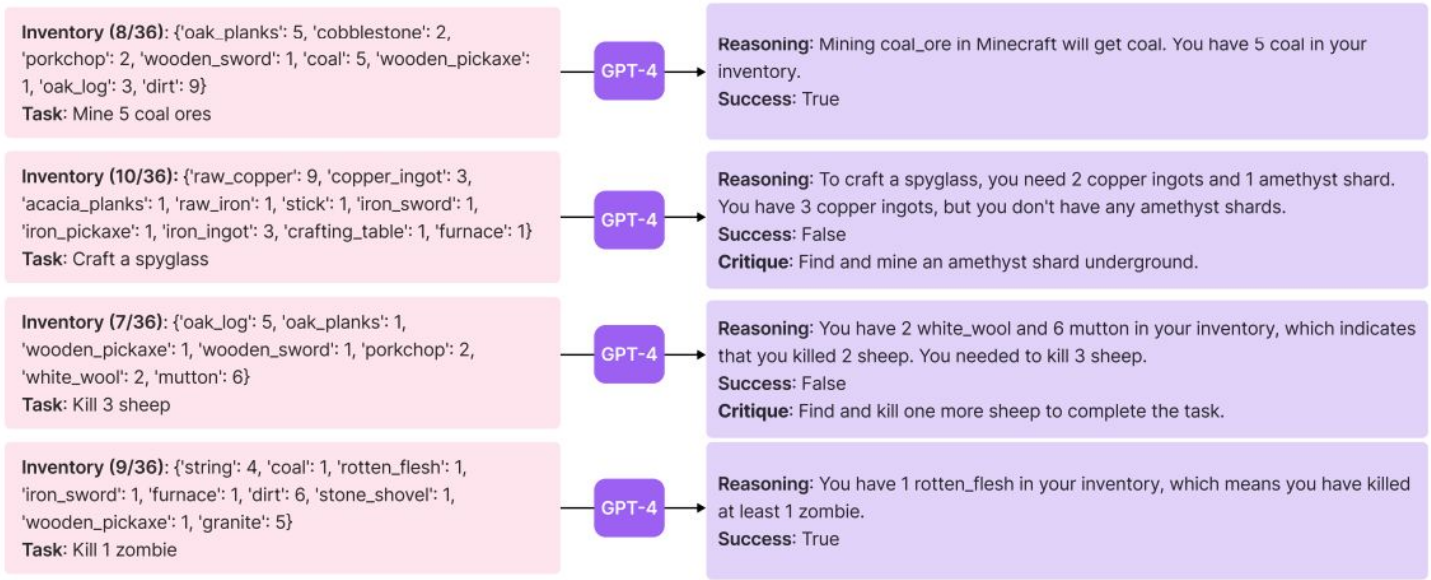


Figure 5: **Left: Environment feedback.** GPT-4 realizes it needs 2 more planks before crafting sticks. **Right: Execution error.** GPT-4 realizes it should craft a wooden axe instead of an acacia axe since there is no acacia axe in Minecraft. We only display the partial prompt for brevity. The full prompt structure for code generation is in Appendix, Sec. A.4.

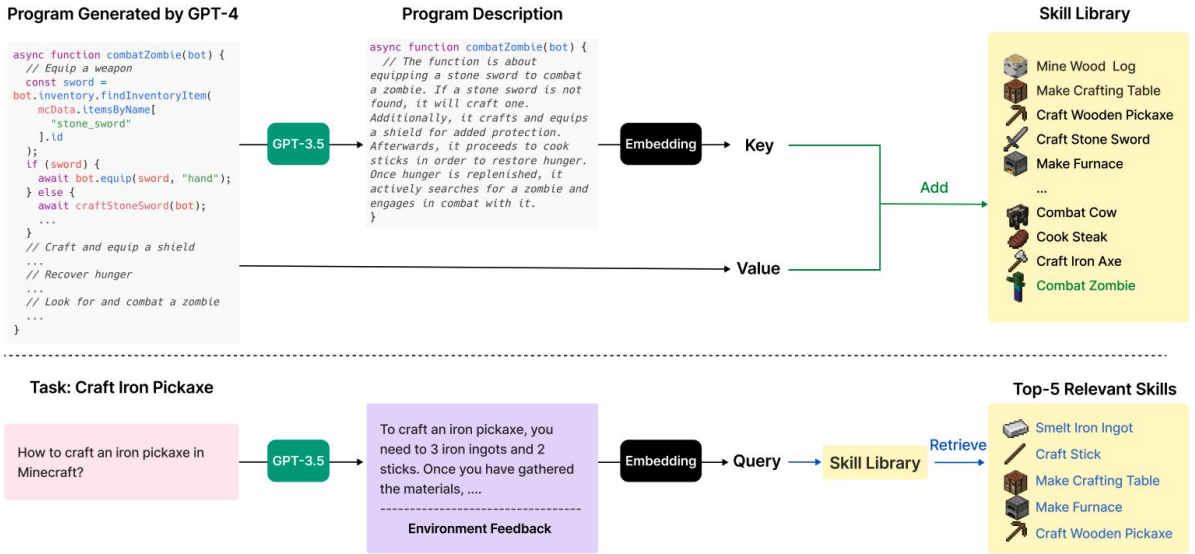
Voyager: Self-verification

- Takes game state and checks for success or failures
  - Success: save to skill library
  - Failure: refine the program base on critique



Voyager: Skill library

- Skills are stored using a text embedding of their description as the key and program code as the value.
- A new task is described (with environment context) and embedded to retrieve the most similar stored skill.



Voyager: Skill Generation

encourage generic skills

Tool Description

(1) **Guidelines for code generation**, such as "`Your function will be reused for building more complex functions. Therefore, you should make it generic and reusable.`";

(2) **Control primitive APIs, and relevant skills** retrieved from the skill library, which are crucial for in-context learning [36–38] to work well;

(3) **The generated code from the last round, environment feedback, execution errors, and critique**, based on which GPT-4 can self-improve (Sec. 2.3);

(4) **The agent's current state**, including inventory, equipment, nearby blocks and entities, biome, time, health and hunger bars, and position;

History

Voyager: Results - Tech Tree Mastery

Table 1: Tech tree mastery. Fractions indicate the number of successful trials out of three total runs. 0/3 means the method fails to unlock a level of the tech tree within the maximal prompting iterations (160). Numbers are prompting iterations averaged over three trials. The fewer the iterations, the more efficient the method.

Method	Wooden Tool	Stone Tool	Iron Tool	Diamond Tool
ReAct [29]	N/A (0/3)	N/A (0/3)	N/A (0/3)	N/A (0/3)
Reflexion [30]	N/A (0/3)	N/A (0/3)	N/A (0/3)	N/A (0/3)
AutoGPT [28]	92 ± 72 (3/3)	94 ± 72 (3/3)	135 ± 103 (3/3)	N/A (0/3)
VOYAGER w/o Skill Library	**7 ± 2** (3/3)	**9 ± 4** (3/3)	29 ± 11 (3/3)	N/A (0/3)
VOYAGER (Ours)	**6 ± 2** (3/3)	**11 ± 2** (3/3)	**21 ± 7** (3/3)	**102** (1/3)

Voyager: Results - Map Coverage

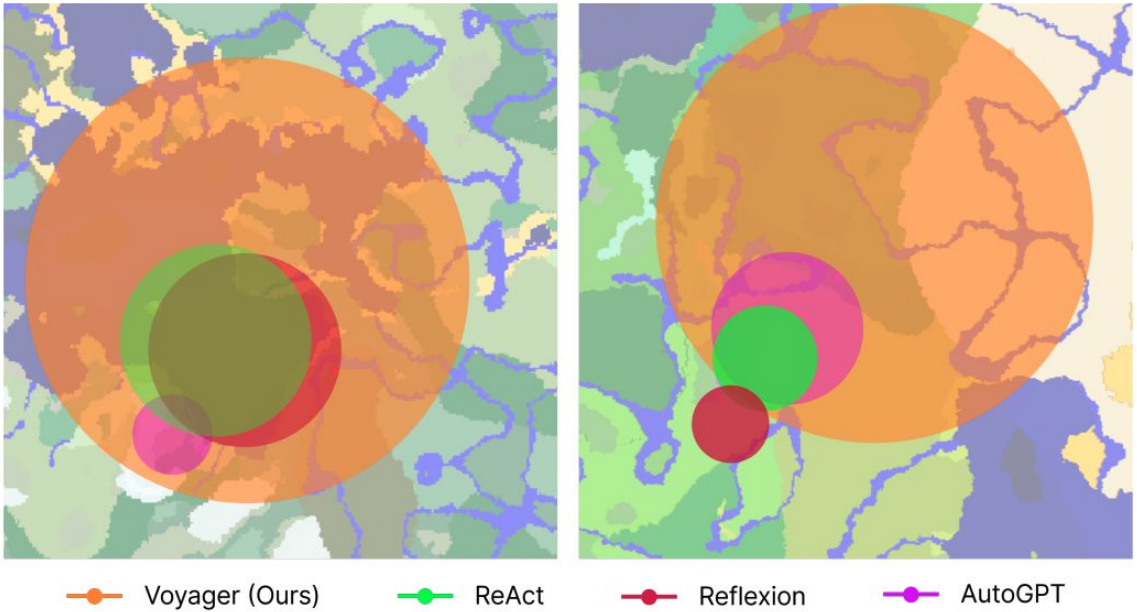


Figure 7: Map coverage: bird's eye views of Minecraft maps. VOYAGER is able to traverse 2.3× longer distances compared to baselines while crossing diverse terrains.

Voyager: Results - Zero-shot

Table 2: Zero-shot generalization to unseen tasks. Fractions indicate the number of successful trials out of three total attempts. 0/3 means the method fails to solve the task within the maximal prompting iterations (50). Numbers are prompting iterations averaged over three trials. The fewer the iterations, the more efficient the method.

Method	Diamond Pickaxe	Golden Sword	Lava Bucket	Compass
ReAct [29]	N/A (0/3)	N/A (0/3)	N/A (0/3)	N/A (0/3)
Reflexion [30]	N/A (0/3)	N/A (0/3)	N/A (0/3)	N/A (0/3)
AutoGPT [28]	N/A (0/3)	N/A (0/3)	N/A (0/3)	N/A (0/3)
AutoGPT [28] w/ Our Skill Library	39 (1/3)	30 (1/3)	N/A (0/3)	30 (2/3)
VOYAGER w/o Skill Library	36 (2/3)	30 ± 9 (3/3)	27 ± 9 (3/3)	26 ± 3 (3/3)
VOYAGER (Ours)	**19 ± 3** (3/3)	**18 ± 7** (3/3)	**21 ± 5** (3/3)	**18 ± 2** (3/3)

Strengths of Voyager

1. Open-Ended Skill Acquisition - lifelong learning

2. Prompt-Based Approach - No Task-Specific Training Needed

3. Skill Library for Memory and Reuse. Unlike agents that start fresh each episode, Voyager remembers what it learns.

4. State-of-the-Art Minecraft Performance. Strong generalization by succeeding in novel tasks in a brand-new world using its learned skill library

5. Effective Use of LLM Knowledge – planner and coder

Weaknesses of Voyager

1. Limited Real-World Perception – text only

2. Quality of LLM-Generated Code – competence is capped by the LLM's coding reliability

3. Scalability of Skill Library. Currently the LLM handles skill selection which might become less reliable as the library grows very large.

4. No Learning of Optimal Policies – no reinforcement learning to statistically improve its policies; it trusts the LLM's judgment.

Future Directions for Voyager

1. Integration of Vision and Multimodal Inputs

2. Open-Source and Efficient Models – fine-tuned on code and Minecraft data

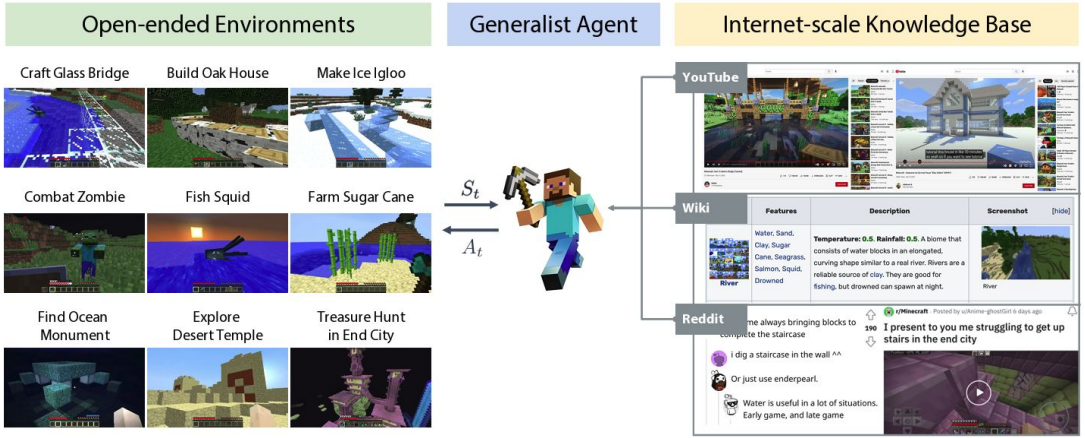
3. Hierarchical Skill Organization – automatically organize skills into categories or hierarchies

4. Extending Memory – knowledge base of world facts it discovers

MINEDOJO: Building Open-Ended Embodied Agents with Internet-Scale Knowledge

Linxi Fan[1], **Guanzhi Wang**[2*], **Yunfan Jiang**[3*], **Ajay Mandlekar**[1], **Yuncong Yang**[4], **Haoyi Zhu**[5], **Andrew Tang**[4], **De-An Huang**[1], **Yuke Zhu**[1 6†], **Anima Anandkumar**[1 2†]
[1]NVIDIA, [2]Caltech, [3]Stanford, [4]Columbia, [5]SJTU, [6]UT Austin
[*]Equal contribution [†]Equal advising
https://minedojo.org

MINEDOJO



- a new framework features a simulation suite with thousands of diverse open-ended tasks and an internet-scale knowledge base

Motivation: Existing embodied agents do not generalize well beyond a narrow set of tasks.

- **Environment of large variety:** enable an unlimited variety of open-ended goals
- **Large-scale learning:** a large-scale database of prior knowledge as learning data.
- **Flexible agent architecture:** an agent that has a unified observation/action space, conditions on natural language task prompts, and adopts the Transformer pre-training paradigm.

MINEDOJO: Tasks

- 32 programmatic tasks
- 32 creative tasks

```
survival_sword_food:
  category: survival
  prompt: survive as long as possible given a sword and some food

harvest_wool_with_shears_and_sheep:
  category: harvest
  prompt: harvest wool from a sheep with shears and a sheep nearby

techtree_from_barehand_to_wooden_sword:
  category: tech-tree
  prompt: find material and craft a wooden sword

combat_zombie_pigman_nether_diamond_armors_diamond_sword_shield:
  category: combat
  prompt: combat a zombie pigman in nether with a diamond sword,
    shield, and a full suite of diamond armors
```

MINEDOJO: Knowledge Bases

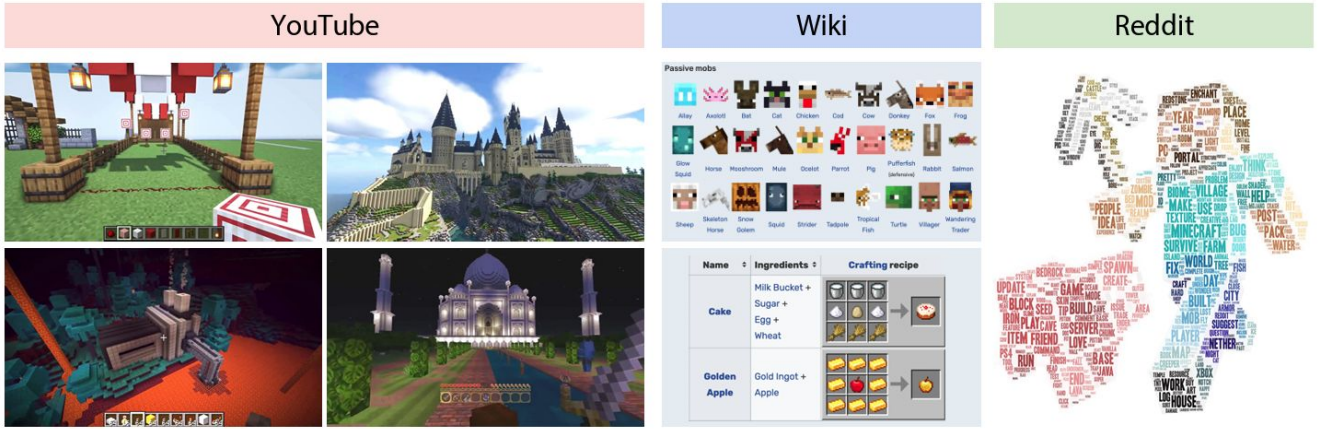
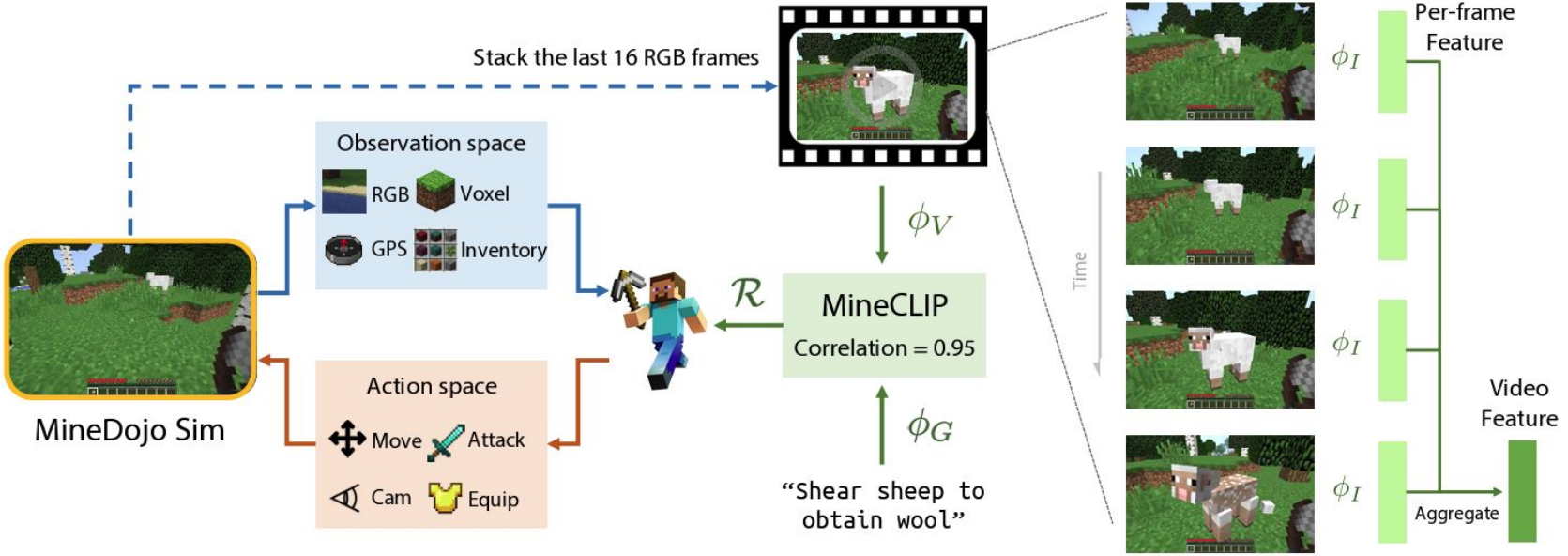


Figure 3: MINEDOJO's internet-scale, multimodal knowledge base. **Left, YouTube videos:** Minecraft gamers showcase the impressive feats they are able to achieve. Clockwise order: an archery range, Hogwarts castle, Taj Mahal, a Nether homebase. **Middle, Wiki:** Wiki pages contain multimodal knowledge in structured layouts, such as comprehensive catalogs of creatures and recipes for crafting. More examples in Fig. A.4 and A.5. **Right, Reddit:** We create a word cloud from Reddit posts and comment threads. Gamers ask questions, share achievements, and discuss strategies extensively. Sample posts in Fig. A.7. Best viewed zoomed in.

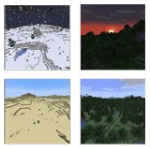
MINEDOJO: Pretraining

- **Reward function from large-scale videos:** $\Phi_{\mathcal{R}} : (G, V) \to \mathbb{R}$
- **RL with the reward function**



MINEDOJO: Main Results

- **MINECLIP agents have stronger zero-shot visual generalization ability**

	Tasks	**Ours** (Attn), train	**Ours** (Attn), unseen test	$\text{CLIP}_{\text{OpenAI}}$, train	$\text{CLIP}_{\text{OpenAI}}$, unseen test
	Milk Cow	64.5 ± 37.1	$\mathbf{64.8 \pm 31.3}(+\ \mathbf{0.8}\%)$	90.0 ± 0.4	$29.2 \pm 3.7\ \ (-67.6\%)$
	Hunt Cow	83.5 ± 7.1	$\mathbf{55.9 \pm 7.2}\ \ (\mathbf{-32.9}\%)$	72.7 ± 3.5	$16.7 \pm 1.6\ \ (-77.0\%)$
	Combat Spider	80.5 ± 13.0	$\mathbf{62.1 \pm 29.7}(\mathbf{-22.9}\%)$	79.5 ± 2.5	$54.2 \pm 9.6\ \ (-31.8\%)$
	Combat Zombie	47.3 ± 10.6	$\mathbf{39.9 \pm 25.3}(\mathbf{-15.4}\%)$	50.2 ± 7.5	$30.8 \pm 14.4(-38.6\%)$

MINEDOJO: Experiments

- **Trained on multi-tasks:**

Group	Tasks	Single Agent on All Tasks	Original	Performance Change
	Milk Cow	**91.5 ± 0.7**	64.5 ± 37.1	↑
	Hunt Cow	46.8 ± 3.7	**83.5 ± 7.1**	↓
	Shear Sheep	**73.5 ± 0.8**	12.1 ± 9.1	↑
	Hunt Sheep	**27.0 ± 1.0**	8.1 ± 4.1	↑
	Combat Spider	72.1 ± 1.3	**80.5 ± 13.0**	↓
	Combat Zombie	27.1 ± 2.7	**47.3 ± 10.6**	↓
	Combat Pigman	**6.5 ± 1.2**	1.6 ± 2.3	↑
	Combat Enderman	0.0 ± 0.0	0.0 ± 0.0	=
	Find Nether Portal	**99.1 ± 0.4**	37.4 ± 40.8	↑
	Find Ocean	**95.1 ± 1.5**	33.4 ± 45.6	↑
	Dig Hole	85.8 ± 1.2	**91.6 ± 5.9**	↓
	Lay Carpet	96.5 ± 0.8	**97.6 ± 1.9**	=

- **MINECLIP shows good zero-shot generalization to significant visual distribution shift.**

	Tasks	Ours (zero-shot)	Ours (after RL finetune)	Baseline (RL from scratch)
	Hunt Pig	1.3 ± 0.6	**46.0 ± 15.3**	0.0 ± 0.0
	Harvest Spider String	1.6 ± 1.3	**36.5 ± 16.9**	12.5 ± 12.7

Comparative Analysis: Voyager, MineDojo, and Related Works

Agent (Year)	Model Arch.	Training Method	Knowledge	Autonomy	Task Diversity & Generalization
MineDojo (Fan et al., 2022)	Env. API + MineCLIP	RL + learned reward - policy learns by maximizing alignment	Explicit: web-scale videos, wiki, Reddit	Episodic autonomy	High task diversity; limited long-horizon
DEPS (Wang et al., 2023)	LLM planner + goal selector	Few-shot prompting + learned goal selector	Implicit via LLM pretraining	Episodic autonomy	70+ tasks zero-shot; multi-tasking
GITM (Zhu et al., 2023)	LLM planner + memory + exec. module	Prompting + memory loop	Explicit: wiki + memory	Episodic autonomy	Full tech-tree; strong generalization
STEVE-1 (Lifshitz et al., 2023)	Text → latent → action model	Offline imitation + latent alignment	Indirect via pretrained data	Episodic autonomy	Early-game tasks; short-horizon generalization

Agent (Year)	Model Arch.	Training Method	Knowledge	Autonomy	Task Diversity & Generalization
Voyager (Wang et al,. 2023)	LLM planner + skill library	Prompting + skill accumulation	Implicit via LLM pretraining	Lifelong, autonomous exploration	Open-ended tasks; skill discovery
Steve-Eye (Zheng et al., 2023)	End-to-end multimodal transformer	Supervised multimodal learning	Explicit via dataset curation	Episodic autonomy	Visual tasks; strategic planning
JARVIS-1 (Wang et al., 2023)	MLLM planner + controllers + memory	Fine-tuning + trained controllers	Explicit: wiki + memory	Lifelong, continual improvement	200+ tasks; long-horizon generalization
Odyssey (Liu et al., 2024)	LLM planner + skill library	Supervised fine-tuning + skills	Explicit: wiki-derived QA	Episodic autonomy	Extended open-world; skill chaining
MP5 (Qin et al., 2024)	Modular planner + active perception	Module-wise supervised/imitation	Implicit via MLLM pretraining	Episodic autonomy	Process-dependent; context-dependent

Strengths of MINEDOJO

1. Internet-Scale Multimodal Knowledge Integration – enabling agents to learn from real human demonstrations.

2. Extensive Task Diversity and Open-Ended Benchmarking. MineDojo defines thousands of programmatic and creative tasks, from survival objectives to aesthetic builds.

3. Generalization via Multimodal Alignment (MineCLIP). By aligning video frames with language using MineCLIP, agents can generalize to unseen tasks described in natural language.

4. Differentiable reward function for any instruction without manual reward engineering.

Weaknesses of MINEDOJO

1. Passive Knowledge Utilization (No Active Reasoning or Retrieval)

2. Limited Performance on Long-Horizon, Multi-Step Tasks: due to the absence of explicit goal decomposition or hierarchical planning mechanisms.

3. Evaluation Bottleneck for Creative/Open-Ended Goals

4. No Memory or Lifelong Learning Capability – each episode is task-specific and isolated

Future Directions for MINEDOJO

1. Integrate Active LLM Planning for Long-Horizon Reasoning

2. Introduce Memory Systems for Lifelong Learning – accumulate knowledge over time, recall past experiences, and refine strategies

3. Enable Active Knowledge Retrieval from External Sources

4. Robust Evaluation Metrics for Creative Tasks – human-in-the-loop evaluation frameworks

5. Expand Multi-Agent and Collaborative Capabilities